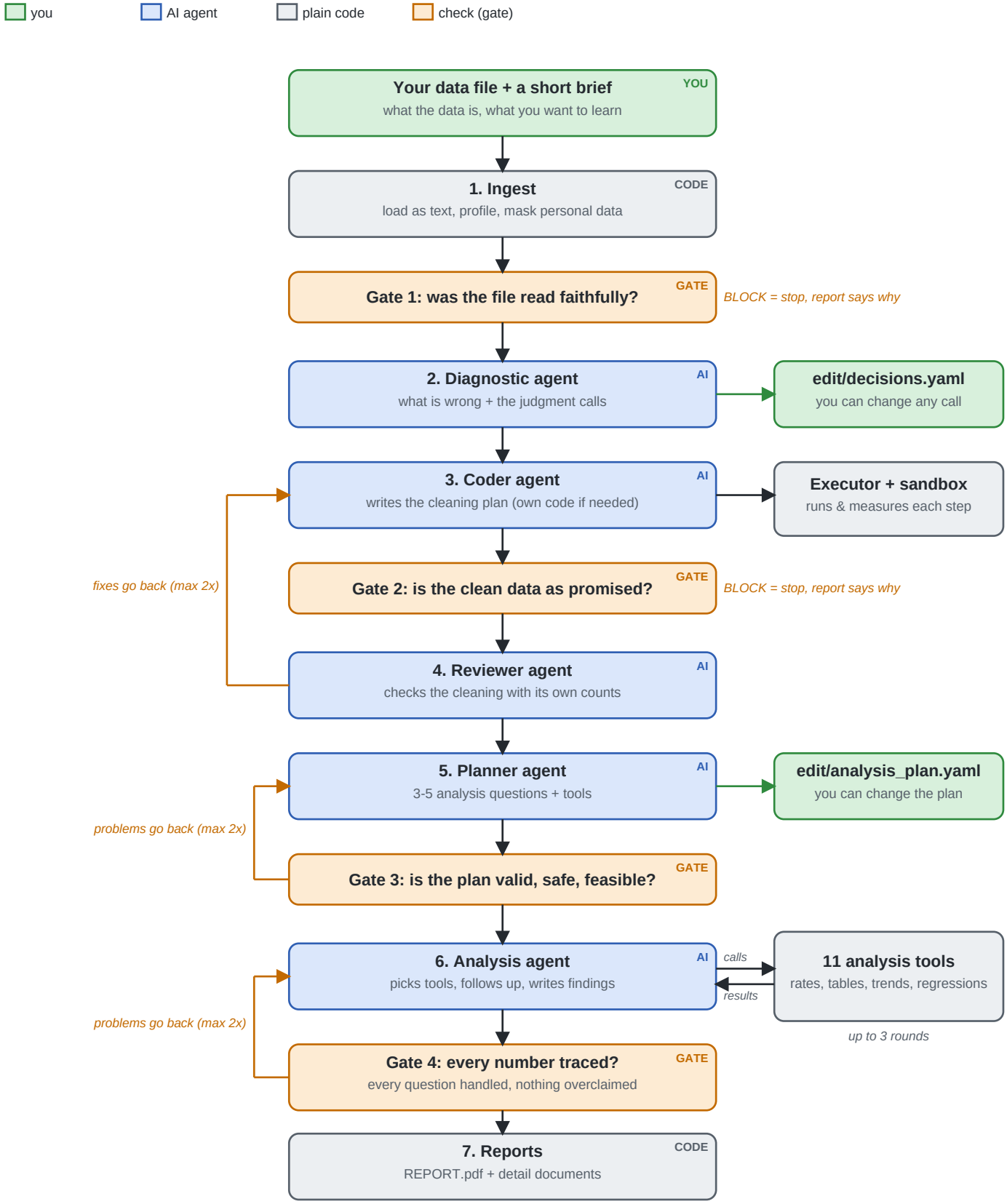
The pipeline at a glance

Auditable agentic data science pipeline - how a data file becomes a checked report



You read REPORT.pdf. To change the result: edit decisions.yaml or analysis_plan.yaml, then run that step again. The original file is never touched, so nothing is lost.

The AI never sees rows or personal data: only masked summaries and totals.

Every change and every number is logged; the original file is never modified.

The idea in short

You give the pipeline a data file and a short brief: what the data is and what you want to learn. It loads and checks the file, cleans it, plans an analysis, runs it, and writes a report. **The AI decides; plain code does the work.** Five AI agents make the judgment calls (what is wrong, how to fix it, what to ask, what the results mean), while tested code loads, changes, measures and calculates. After each AI step a **gate**, a set of fixed checks in plain code, decides whether the result may go on.

Three rules hold everywhere: the AI never sees rows or personal data, only masked summaries and totals; every change and every number is logged, and the original file is never modified; and nothing is decided silently - every choice is in the report with its reasons, and you can change it and run again.

Gates: what the outcomes mean

Outcome	Meaning
PASS	Everything checked out; the run continues.
FLAG	The run continues, and the issue is listed in the report's wrap-up.
BLOCK	The run stops at this step. For an AI step, the problems first go back to the agent to fix (at most twice).

The steps

1. Ingest [code]

Purpose	**Read the file exactly as it is, and describe it.**
Gets	Your data file (CSV, Excel, Parquet, JSON) and the brief.
Does	Loads every value as text, so nothing is silently converted. Profiles each column (type, missing values, odd values), detects personal data and masks it, and checks the claims in your brief (e.g. 'the ID is unique').
Gives	A masked summary - the only view of the data the AI ever gets - and a fingerprint of the original data.
Safeguards	**Gate 1**: every row accounted for, under 1% of lines unreadable, unique column names, every type recognised or flagged. No AI is involved, so a blocked file costs nothing.

2. Diagnostic agent [AI]

Purpose	**Find what is wrong with the data, and the questions the data cannot answer by itself.**
Gets	The masked summary and the brief.
Does	Lists **issues** (clear errors with a clear fix, e.g. 'NA' meaning missing, Danish decimal commas) and **judgment calls** (choices either way could be right, e.g. 'does 999 mean unknown?'). It decides each call from your brief, taking the option that changes the least when the brief does not settle it.
Gives	The diagnosis, and **edit/decisions.yaml** where you can change any decision.
Safeguards	Fixed checks: every column covered, the target never filled in, excluded columns kept out, every needed type conversion planned. Problems go back to the agent, at most twice.

3. Coder agent + executor [AI + code]

Purpose	**Turn the diagnosis into cleaning steps, and carry them out safely.**
Gets	The masked summary, the diagnosis and the decisions.
Does	Writes a cleaning plan from 15 ready-made operations, or its own Python when none fits. The **executor** (code) runs each step on a copy, starting from the original data, and measures exactly what changed; AI-written code runs in a locked sandbox and may only change what it declared.
Gives	The clean data, a log of every change, and a schema contract describing the clean data.
Safeguards	**Gate 2**: every column has the promised type, the ID is still unique, the target was never filled in, rows lost under 5% pass (5-20% flag, over 20% block), and replaying the plan from the original gives the identical result.

4. Reviewer agent [AI]

Purpose	**A second opinion: is the cleaning actually right?**
Gets	The brief, the diagnosis, every step with its measured effect, and Gate 2's results.
Does	Asks its own counting questions of the data (only totals come back, never rows), then accepts, sends specific fixes back to the coder (at most twice), and lists questions only you can answer.
Gives	A verdict, and its questions for you in the report.
Safeguards	A claim citing a number it was never shown is thrown out; it needs two claims verified with numbers to accept. Optional (--no-review).

5. Planner agent [AI]

Purpose	**Decide what is worth finding out.**
Gets	Your brief (above all the goal), a totals-only summary of the clean data, the list of tools, and columns that predict the outcome suspiciously well.
Does	Writes 3-5 questions tied to your goal, each with the columns and tools to answer it; says how it read your brief; leaves out columns with reasons; marks questions this data cannot answer, with the reason.
Gives	The plan, in **edit/analysis_plan.yaml**. It runs straight on; edit it and re-run to change it, and your version wins.
Safeguards	**Gate 3**: every column and tool exists, the target is yours, nothing personal or excluded is analysed, a column that gives the answer away is left out, and every question has enough rows or is marked not possible.

6. Analysis agent + tools [AI + code]

Purpose	**Answer the plan's questions, and follow what is interesting.**
Gets	The brief, the plan, the data summary and the cleaning limitations.
Does	In up to three rounds it asks for tool calls, reads the results, and follows up on what stands out (e.g. 'Cherbourg looks good - is that the port or its passengers?'). The tools (code) compute everything. Then it writes an answer per question, findings with the tool results they rest on, its own doubts, and recommendations.
Gives	The write-up, and every tool result and finding saved with references to how it was made.
Safeguards	**Gate 4**: every plan question answered or marked not possible, every finding rests on calls that ran, every number in the text is one a tool reported (rounding allowed, numbers the AI worked out itself are not), and a model no better than no model at all cannot back a 'strong' finding.

7. Reports [code]

Purpose	**Tell the whole story in a form that is easy to read.**
Gets	Everything in the run folder.
Does	Builds REPORT.pdf (3-4 pages: in brief, what we found, how good the data is, the decisions made for you, and a self-critical wrap-up) and four detail documents behind it.
Gives	REPORT.pdf plus details/1_analysis, 2_data_quality, 3_decisions and 4_audit.
Safeguards	Every number comes from the logs; AI-written text is marked; no personal data appears.

The agents side by side

Agent	Decides	Writes code?	Checked by
Diagnostic	What is wrong; the judgment calls	No	Diagnosis checks
Coder	How to clean it	Sometimes, in a sandbox	Executor + Gate 2
Reviewer	Whether the cleaning is right	No (fixed counting queries)	Its own verified numbers
Planner	What to find out	No	Gate 3
Analysis	Which tools to run; what it means	No (picks tools)	Gate 4

The analysis tools

The analysis agent cannot calculate anything itself; it chooses from these tools, and every result is aggregated (groups under 10 rows are hidden).

Tool	Answers
summary_table	The basic numbers for every column
distribution	How one column's values are spread
missingness	Where values are missing, and whether that relates to the outcome
correlation	Which numbers move together
target_by_group	The outcome per group or range, with 95% intervals
target_by_two_groups	The outcome per combination of two columns, e.g. sex within class
target_by_ratio	The outcome by one number relative to another, e.g. loan / income
trend_over_time	The outcome per year, quarter or month
compare_by_target	How a number differs between the two outcomes
linear_regression	Effects on a number outcome, the others held fixed
logistic_regression	Effects on a yes/no outcome, the others held fixed
subset / cuts	Any tool on one subgroup only; your own ranges, e.g. under 16

What a run leaves behind

Where	What
REPORT.pdf	Start here: the essentials in 3-4 pages
details/	One document per section, the Excel column overview, the charts
edit/	decisions.yaml and analysis_plan.yaml: change these, then re-run
data/	The clean data, the removed rows and the unreadable lines
logs/	The machine-readable audit trail: every AI attempt, step, check and result